Bee_Fi Codebase Guide

Multi-Agent Reinforcement Learning for Orbital Task Allocation

Table of Contents

High-Level System Overview

This is a **Multi-Agent Reinforcement Learning (MARL) system** simulating orbital satellite "bees" performing coordinated task harvesting. It combines:

- **Orbital mechanics** (Keplerian orbits + optional SGP4 propagation)
- **PPO training** (Proximal Policy Optimization)
- **Battery-driven task reassignment** (fault tolerance)
- **Decentralized communication** (retask boards between bees)

Goal: Train agents to maximize pollen collection while managing battery constraints, respecting time windows, and handling dynamic task reassignment when teammates fail.

Core Concepts

1. The Environment (`bees_env.py`)

Agents (Bees)

- **5 agents** moving on elliptical orbits around a central point
- Each bee has:
  - **Position**: (x, y, z) Cartesian coordinates
  - **Battery**: Current charge (0-100%) → dies when empty
  - **Load**: Current pollen (0-10.0 units)
  - **Capacity**: Max pollen per bee (10.0 units)
  - **Assigned flowers**: Personal task queue

Tasks (Flowers)

- **12 static tasks** at fixed coordinates

- Each has:
  - Pollen amount (0.8-10.0 units)
  - Priority level
  - **Time windows** (HARD/SOFT/NONE):
    - **HARD**: One-time only, severe penalty (-50) if missed
    - **SOFT**: Repeating opportunities every N steps
    - **NONE**: Anytime harvest allowed
- Only one bee can harvest a flower per window

Actions per Bee

```
0 = DONOTHING   → Continue orbiting (no reward/penalty)
1 = HARVEST     → Collect pollen from nearby flower (+reward if successful)
2 = GROOM       → Offload pollen OR trigger recharge (+reward if strategic)
```

Action Availability Mask:

- Can't harvest if load at capacity
- Can't harvest if flower already assigned to another bee
- Can't groom if load < 1.0 unit

2. Battery & Task Reassignment (The Novel Part)

When a bee's battery reaches 0:

```
Bee Battery = 0
    ↓
AUTOMATIC TASK RELEASE
    ↓
All assigned flowers return to pool (assigned_bee = None)
    ↓
Broadcast to nearest active bee via retask_board
    ↓
Other bees can claim from pool after recharge
```

This creates **decentralized fault tolerance**: when a bee dies, its tasks don't vanish—they're redistributed to healthy bees through a **decentralized retask board**.

Retask Board (per-bee):

- Each bee has a small list (default size=3) of orphaned tasks it can claim
- Created when nearby bee's battery dies
- Holds metadata: flower_id, pollen, priority, window info
- Cleared after bee recharges and claims what it needs

3. Reward Structure

Action	Reward	Condition
Harvest success	+5.0 + pollen bonus	Successful collection
Pollen bonus	up to +10	Higher pollen = higher reward
Strategic groom	+5.0 to +15.0	Load ≥80%, good timing
Strategic recharge	+8.0 to +13.5	Low battery (<20%), proactive
Unnecessary groom	**-0.3**	Groom when load low
Invalid harvest	**-0.05**	Attempt on unavailable target
Miss HARD window	**-50**	Critical penalty
Time decay	**-0.01/step**	Encourages task completion

Key insight: Rewards incentivize **proactive** behavior:

- Recharge before emergency
- Groom when full (don't waste capacity)
- Plan ahead for HARD windows

4. Observation Space (Per Bee)

Each bee's observation includes:

```
{
  "position": [x, y, z],              # Current 3D coordinates
  "status": [battery%, load%, mode],  # State vector
  "flowers": [[info] × 12],           # All flowers' features:
                                       #   - position
                                       #   - pollen
                                       #   - priority
                                       #   - current_harvester_id
                                       #   - window_status
  "step_count": [current_step],       # Temporal awareness
  "consensus": [agreement_vector],    # Other bees' task assignments
  "retask_board": [orphaned_tasks],   # Nearby failed bee's tasks
  "action_availability": [mask],      # Which actions are valid
}
```

Neural Network Architecture (`bee_policy.py`)

Actor Network (Decision Making)

Architecture:

```
Input (per-bee observation)
    ↓
Embedding layer (position, battery, load)
    ↓
Multi-Head Attention over flowers
    (learns: which flowers are priority, who needs them, timing urgency)
    ↓
Transformer block ×2
    ↓
Action logits (3 actions: DONOTHING, HARVEST, GROOM)
    ↓
Softmax → action distribution
```

Key Feature: Attention Mechanism

- Each bee attends to all flowers
- Learns to focus on:
  - Nearby flowers (low distance)
  - High-pollen flowers
  - Flowers with hard deadlines
  - Unassigned flowers (high priority)
- Attention weights are **interpretable** (can see what bee is "thinking")

Centralized Critic Network (Value Function)

Architecture:

```
Global state (all bees' positions, batteries, loads + all flowers)
    ↓
FC layer (1024 units)
    ↓
ReLU activation
    ↓
FC layer (512 units)
    ↓
Value output V(s) ∈ [0, ∞)
```

Why Centralized Critic?

- Sees **global state** of all bees
- Estimates value V(s) for computing GAE advantages
- Helps stabilize training by understanding coordinated effects
- Reduces variance in value estimates

Key Point: At inference time, only the **decentralized actor** is used. The critic only helps during training.

Training Loop (`train_orbital_v2.py`)

PPO Algorithm Overview

Proximal Policy Optimization (PPO) is chosen because:

- Stable convergence (lower variance than vanilla policy gradient)
- Sample-efficient (reuses trajectory data)
- Trust-region optimization prevents catastrophic forgetting
- Works well with continuous action spaces

Training Pseudocode

```
Initialize actor network π(a|s), critic network V(s)

For epoch = 1 to 1500:

    Rollout collection (16 parallel environments):
        For each environment:
            For t = 0 to 511:  # 512-step rollout
                Get observation s_t (5 bees × 12 flowers)
                Sample action a_t ~ π(a|s_t)  # from actor
                Step environment → s_{t+1}, r_t
                Store (s_t, a_t, r_t, log_prob, V(s_t))

    Compute advantages using GAE (Generalized Advantage Estimation):
        TD error: δ_t = r_t + γ·V(s_{t+1}) - V(s_t)
        Advantage: A_t = δ_t + (γλ)·δ_{t+1} + ...
        Return: G_t = A_t + V(s_t)

        Where γ=0.99 (discount), λ=0.95 (GAE parameter)

    Optimize actor & critic (3 PPO epochs with minibatches):

        Actor loss = -log π(a|s)·A + entropy_coef·H(π)
            • Gradient ascent on advantages
            • Entropy bonus encourages exploration
            • Log-prob prevents taking too big a step

        Critic loss = (G_t - V(s_t))²
            • MSE between estimated and true value
            • Helps actor know which states are good/bad

        PPO clipping (prevent too-large updates):
            If π_new(a|s) / π_old(a|s) > 1 + clip_ε:
                Clip to 1 + clip_ε
            Prevents overfitting to one batch

    Save checkpoints:
        If mean_reward_100 > best_reward:
            Save best_actor.pt, best_critic.pt
        Every 100 updates:
            Save upd{N}_actor.pt, upd{N}_critic.pt
```

```
Log to TensorBoard:
    • reward/episode - raw episode reward
    • reward/mean_100 - smoothed running average
    • loss/policy - actor loss
    • loss/value - critic loss
    • loss/entropy - exploration bonus
```

Hyperparameters

Parameter	Value	Notes
Rollout length	512 steps	Per environment before update
Parallel envs	16	Simultaneous data collection
Batch size	128	Minibatch during PPO update
PPO epochs	3	Reuse data 3 times
Discount γ	0.99	99% future reward weight
GAE λ	0.95	Bias-variance tradeoff
Clip ε	0.2	Trust region size (±20%)
Entropy coef	0.01	Exploration bonus strength
Value coef	0.5	Critic loss weight
Learning rate	3e-4 (actor), 1e-3 (critic)	Adam optimizer
Total updates	1500	~2.5 hours on RTX 5090

Curriculum Learning

To force learning of robust behaviors:

- **70% of episodes**: Normal battery (250-450 steps capacity)
    - Battery drains at normal rate (0.1-0.15% per step)
    - Allows relaxed task planning
- **30% of episodes**: Low-battery challenge (83-150 steps capacity)
    - Battery drains 2-3x faster
    - Forces learning of:
        - Early recharge behavior
        - Quick decision-making
        - Task prioritization
        - Handling sudden failures

This curriculum prevents the agent from learning lazy strategies ("always harvest immediately") that fail in emergency.

Telemetry Integration (`telemetry_mapper.py`)

Converts real satellite telemetry (JSON) → simulation objects

Input Format

```
{
  "satellites": [
    {
      "id": "SAT-1",
      "x": 1000, "y": 2000, "z": 500,
      "vx": 5.0, "vy": 10.0, "vz": 2.0,
      "battery_charge": 85.0,
      "battery_capacity": 100.0,
      "current_load": 2.5
    }
  ],

  "tasks": [
    {
      "id": "TASK-1",
      "x": 100, "y": 200,
      "pollen": 5.0,
      "priority": "HIGH",
      "window_type": "HARD",
      "window_start": 100,
      "window_end": 200
    }
  ],

  "failed_satellites": ["SAT-3", "SAT-7"],

  "task_reassignment": {
    "SAT-3": ["TASK-5", "TASK-8"],
    "SAT-7": ["TASK-2"]
  }
}
```

Mapping Logic

Telemetry	Simulation	Notes
Satellites	Bees	ID preserved, positions mapped
Tasks	Flowers	Windows converted to time constraints
Failed satellites	Terminated agents	Orchaned tasks added to retask boards
Task reassignment	Retask board	Failed bee's tasks queued for claim
Battery %	Bee battery	Direct mapping

Telemetry	Simulation	Notes
Load	Bee load	Current pollen amount

Evaluation Workflow

```
Load telemetry JSON
    ↓
Map to BeeForagingEnv
    ↓
Run trained policy (1 episode)
    ↓
Logs: harvests, retasking events, comms
    ↓
Output metrics:
    - Total reward
    - Flowers harvested / Total flowers
    - Battery deaths
    - Task completion rate
    - Missed HARD windows
```

Key Files & Roles

File	Size	Purpose
bee_state.py	542 lines	Bee/Flower classes, Keplerian orbit dynamics, optional SGP4 TLE propagation
bees_env.py	800+ lines	PettingZoo environment, step logic, battery/retasking physics
bee_policy.py	300+ lines	Actor (attention) + Critic networks, action sampling
train_orbital_v2.py	549 lines	PPO training loop, experience replay, model checkpointing
train_utils.py	100+ lines	Config loading, model save/load utilities
bee_orbits_3d.py	500+ lines	3D matplotlib visualization, HUD, real-time monitoring, animation export
telemetry_mapper.py	400+ lines	Real telemetry JSON → simulation conversion
test_model_telemetry.py	300+ lines	Evaluate trained model on real scenarios, baseline comparison

File	Size	Purpose
baseline_comparison.py	200+ lines	Classical policies: Random, Greedy, Hungarian algorithm
config.yaml	20 lines	Environment & training hyperparameters
requirements.txt	20 lines	Dependencies: torch, numpy, gymnasium, pettingzoo, etc.

Detailed File Descriptions

`bee_state.py`

Defines the **Bee** and **Flower** classes:

Bee class:

- Orbital parameters (semi-major axis, eccentricity, inclination, etc.)
- Battery state machine (IDLE → HARVESTING → GROOMING → RECHARGING)
- SGP4 TLE support (optional): Load real satellite elements, propagate with high accuracy
- Methods: `step()`, `harvest()`, `groom()`, `recharge()`

Flower class:

- Static position, pollen amount
- Time window (HARD/SOFT/NONE)
- Assigned bee tracker

`bees_env.py`

PettingZoo ParallelEnv subclass:

- **reset()**: Initialize 5 bees on orbits, 12 flowers in space
- **step()**: Execute actions, compute rewards, update battery/tasks
- **get_obs()**: Build observation dicts for all bees
- Task assignment logic (initial round-robin, then dynamic via retask board)
- Battery drain + recharge + death logic
- Communication chain simulation (retask board propagation)

`bee_policy.py`

Neural networks:

Actor:

```
class Actor(nn.Module):
    def __init__(self, obs_size, action_size):
        self.embedding = nn.Linear(obs_size, 256)
        self.attention = MultiHeadAttention(256, num_heads=4)
        self.transformer = TransformerBlock(256)
        self.head = nn.Linear(256, action_size)
```

```
    def forward(self, obs_dict):
        # Embed flowers
        flower_embeds = self.embedding(obs_dict["flowers"])
        # Attend to flowers
        attended = self.attention(flower_embeds)
        # Transformer blocks
        x = self.transformer(attended)
        # Action logits
        return self.head(x)
```

Critic:

```
class CentralizedCritic(nn.Module):
    def __init__(self, global_state_size):
        self.fc1 = nn.Linear(global_state_size, 1024)
        self.fc2 = nn.Linear(1024, 512)
        self.value_head = nn.Linear(512, 1)

    def forward(self, global_state):
        x = F.relu(self.fc1(global_state))
        x = F.relu(self.fc2(x))
        return self.value_head(x)
```

`train_orbital_v2.py`

Main training script:

- Argument parsing: config, output dir, seed, CPU/GPU, resume
- Seed setting for reproducibility
- PPO training loop (pseudocode above)
- Model checkpointing (best, periodic, final)
- TensorBoard logging
- Resume functionality (continue from checkpoint)

`train_utils.py`

Utilities:

```
def load_config(path: str) -> dict:
    """Load YAML config"""

def save_models(actor, critic, output_dir, tag):
    """Save PyTorch models with tag"""

def load_models(output_dir, tag, device):
    """Load actor & critic from checkpoint"""
```

bee_orbits_3d.py

Real-time visualization:

- **3D plot**: Orbital ellipses + bee positions (color-coded by battery %)
- **HUD overlay**:
  - Bee stats: battery, load, assigned flowers
  - Global stats: completion %, total reward
  - Event log: harvests, retasking, comms
- **Animation**: Render full episode as video
- **Export**: Save as MP4 for presentations

telemetry_mapper.py

Conversion logic:

```
def load_telemetry(json_path: str) -> dict:
    """Parse real telemetry JSON"""

class TelemetryMapper:
    def __init__(self, telemetry_dict):
        self.satellites = telemetry_dict["satellites"]
        self.tasks = telemetry_dict["tasks"]
        self.failed_sats = telemetry_dict["failed_satellites"]

    def to_env(self) -> BeeForagingEnv:
        """Create environment from telemetry"""
```

test_model_telemetry.py

Evaluation pipeline:

```
For each telemetry scenario:
    1. Load real data
    2. Create environment
    3. Run trained model (or baseline)
    4. Compute metrics:
       - Total reward
       - Task completion rate
       - Efficiency (reward / steps)
       - Battery deaths
    5. Log results to JSON
```

baseline_comparison.py

Classical algorithms:

- **Random**: Random actions
- **Greedy**: Always harvest closest unassigned flower
- **Hungarian**: Optimal assignment using cost matrix + Hungarian algorithm
    - Slower but optimal for assignment problem

Useful for validating that RL beats baselines.

Execution Flow

1. Training Workflow

```
$ python train_orbital_v2.py --config config.yaml --seed 42

[train_orbital_v2] Starting training for 1500 updates...
[ENV] Bee 0 harvested flower 5 (+8.0pollen, reward: 6.2, ...)
...
[upd 50] reward=145.3, mean100=142.1, loss=0.234
[BEST] New best mean reward: 142.1
...
[upd 1500] TRAINING COMPLETE
  Best mean reward: 287.4
  Models saved to: outputs/
```

Outputs:

- `outputs/best_actor.pt` - Best actor policy
- `outputs/best_critic.pt` - Best critic
- `outputs/upd{N}_actor.pt`, `outputs/upd{N}_critic.pt` - Checkpoints
- `outputs/events.out.tfevents.*` - TensorBoard logs

2. Visualization/Inference

```
$ python bee_orbits_3d.py --policy --model_tag best

Loading: outputs/best_actor.pt
Running episode with trained policy...
[Rendering 3D orbits + HUD]
[EPISODE END] Reward: 287.4, Steps: 342/800, Flowers: 12/12
Saving video to: outputs/orbital_animation.mp4
```

3. Real Telemetry Evaluation

```
$ python test_model_telemetry.py --telemetry telemetry.json

Loading telemetry...
Running model...
```

```
[SCENE 1] Reward: 245.3, Completion: 11/12, Deaths: 1
[SCENE 2] Reward: 312.1, Completion: 12/12, Deaths: 0
Average reward: 278.7
Results saved: telemetry_results.json
```

4. Baseline Comparison

```
$ python baseline_comparison.py --telemetry telemetry.json

Model: RL Policy
  Reward: 278.7, Completion: 11.5/12 avg, Deaths: 0.3 avg

Baseline: Random
  Reward: 120.4, Completion: 7.2/12 avg, Deaths: 3.4 avg

Baseline: Greedy
  Reward: 195.6, Completion: 9.8/12 avg, Deaths: 1.2 avg

Baseline: Hungarian
  Reward: 245.3, Completion: 11.9/12 avg, Deaths: 0.1 avg

RL beats greedy by 42.6%, but Hungarian is still strong (optimal
assignment).
Hungarian doesn't handle dynamic failures well; RL adapts via retasking.
```

What Makes This Unique

1. **Battery → Task Reassignment**

Traditional MARL assumes static task assignments. This system models **realistic satellite failure**: when a bee dies, its tasks are automatically distributed through a decentralized broadcast mechanism.

2. **Decentralized Retask Board**

No shared global state. Each bee has a local list of orphaned tasks from nearby failures. This:

- Scales to larger swarms (info spreads locally, not globally)
- Handles partial network failures
- Incentivizes cooperative behavior without explicit coordination

3. **Time Windows (HARD/SOFT)**

Most swarm tasks assume "anytime" servicing. Here:

- **HARD windows** create urgent constraints (penalty -50 for missing)
- **SOFT windows** allow repeat chances
- Tests agent's ability to plan ahead and prioritize

4. **Curriculum Learning**

Battery-constrained episodes (30%) force learning of:

- Reactive recharge behavior
- Quick decision-making under stress
- Graceful degradation when teammates fail

Standard MARL training can learn lazy strategies (always harvest first). Curriculum prevents this.

5. **Real Telemetry Integration**

Unlike toy simulations, this system can:

- Load actual satellite positions (from TLE or API)
- Map real task locations
- Test on realistic failure scenarios
- Compare vs classical algorithms (Hungarian, Greedy)

6. **Interpretable Attention**

The actor's attention weights over flowers show:

- What the bee is "looking at"
- Why it makes decisions
- Which flowers are considered priority

This interpretability is important for safety-critical applications (satellites).

7. **Multi-Agent Credit Assignment**

The centralized critic sees global state, allowing it to understand:

- Cooperative effects (when two bees work together efficiently)
- Competitive effects (congestion at popular flowers)
- Collective success metrics (all flowers harvested = +bonus)

Training Insights

Convergence Behavior

- **First 100 epochs**: Agent learns basic harvest behavior, task completion rises from 0% to 40%
- **Epochs 100-500**: Discovers battery management, completion rises to 75%
- **Epochs 500-1500**: Refines retasking behavior, learns proactive recharge, reaches 95%+ completion

Common Failure Modes (Early Training)

- **Out of Battery**: Agent doesn't recharge early enough → dies mid-episode
- **Congestion**: Multiple bees chase same flower → inefficient
- **Missed Hard Windows**: Agent doesn't plan ahead for deadlines

Solutions Learned (Late Training)

- **Predictive Recharge**: Bee charges at 30% battery, not waiting until 0%
- **Load Balancing**: Attention learns to spread bees across flowers
- **Deadline Awareness**: Bee prioritizes high-deadline tasks

---

Troubleshooting & Extensions

Common Issues

Q: Training is slow (>10s/update)

- GPU not being used: Check `--cpu` flag not set
- Parallel envs bottlenecked: Reduce `rollout_len` in config
- Solution: Use `torch.cuda.is_available()` to verify GPU

Q: Model doesn't learn (reward stuck at 50)

- Learning rate too high: Reduce to 1e-4
- Entropy decay too fast: Increase `entropy_coef` to 0.05
- Solution: Restart with `--seed` for reproducibility, check TensorBoard loss curves

Q: Retasking not triggering

- Battery threshold not low enough: Reduce `battery_min_steps`
- Retask board size too small: Increase in config
- Solution: Enable debug logging, check task pool

Possible Extensions

1. **Heterogeneous agents**: Bees with different speeds, capacities, missions
2. **Communication cost**: Retasking uses bandwidth (penalty for excessive broadcast)
3. **Collision avoidance**: Add obstacles, test safe separation
4. **Continuous actions**: Instead of discrete GROOM/HARVEST, continuous throttle
5. **Larger swarms**: Scale from 5 to 50+ bees, test swarm coordination limits
6. **Hierarchical RL**: High-level coordinator assigns regions, low-level agents plan locally

---

References & Credits

- **PPO Algorithm**: Schulman et al., "Proximal Policy Optimization Algorithms" (OpenAI, 2017)
- **SGP4 Propagation**: Vallado, Crawford, Hujsa, "Revisiting Spacetrack Report #3" (2006)
- **PettingZoo**: Multi-agent environment library (Farama Foundation)
- **PyTorch**: Deep learning framework

Last Updated: February 5, 2026
Author: Bee_Fi Development Team
Status: Active Development